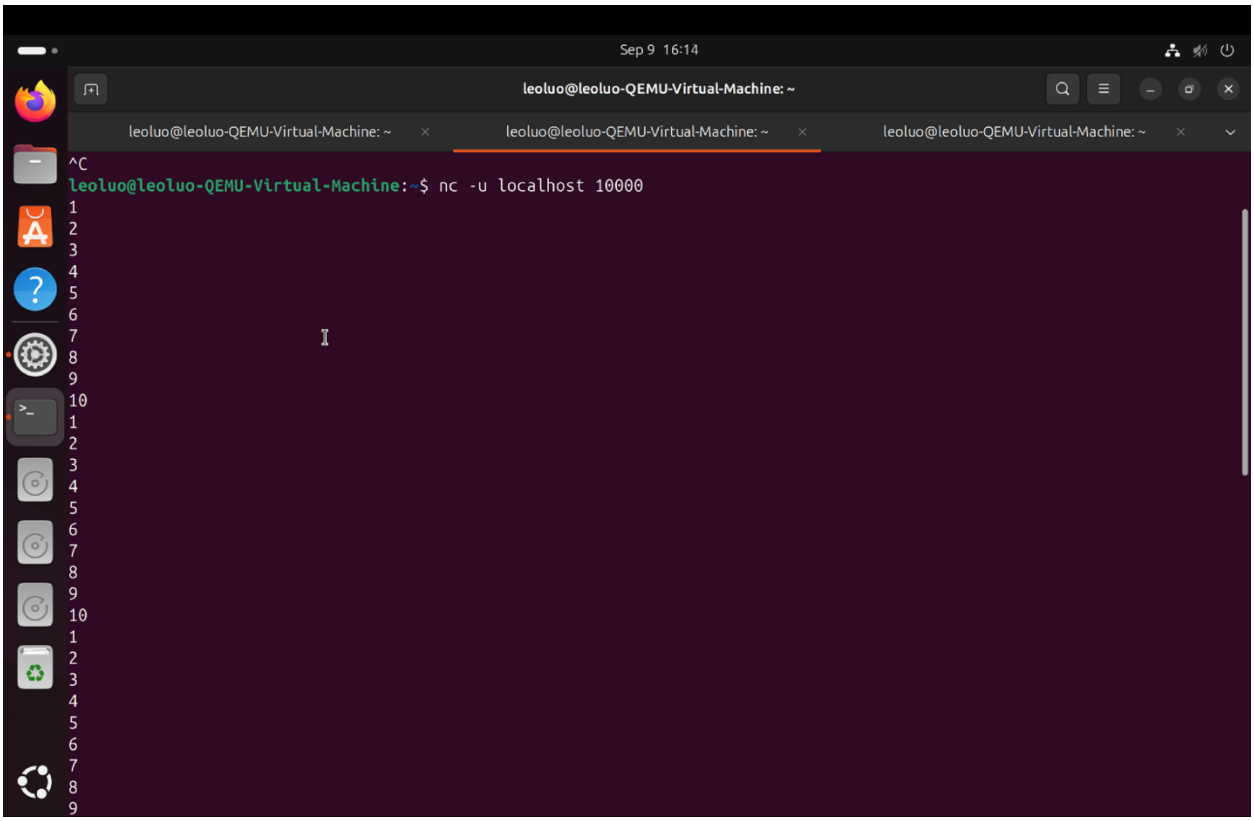
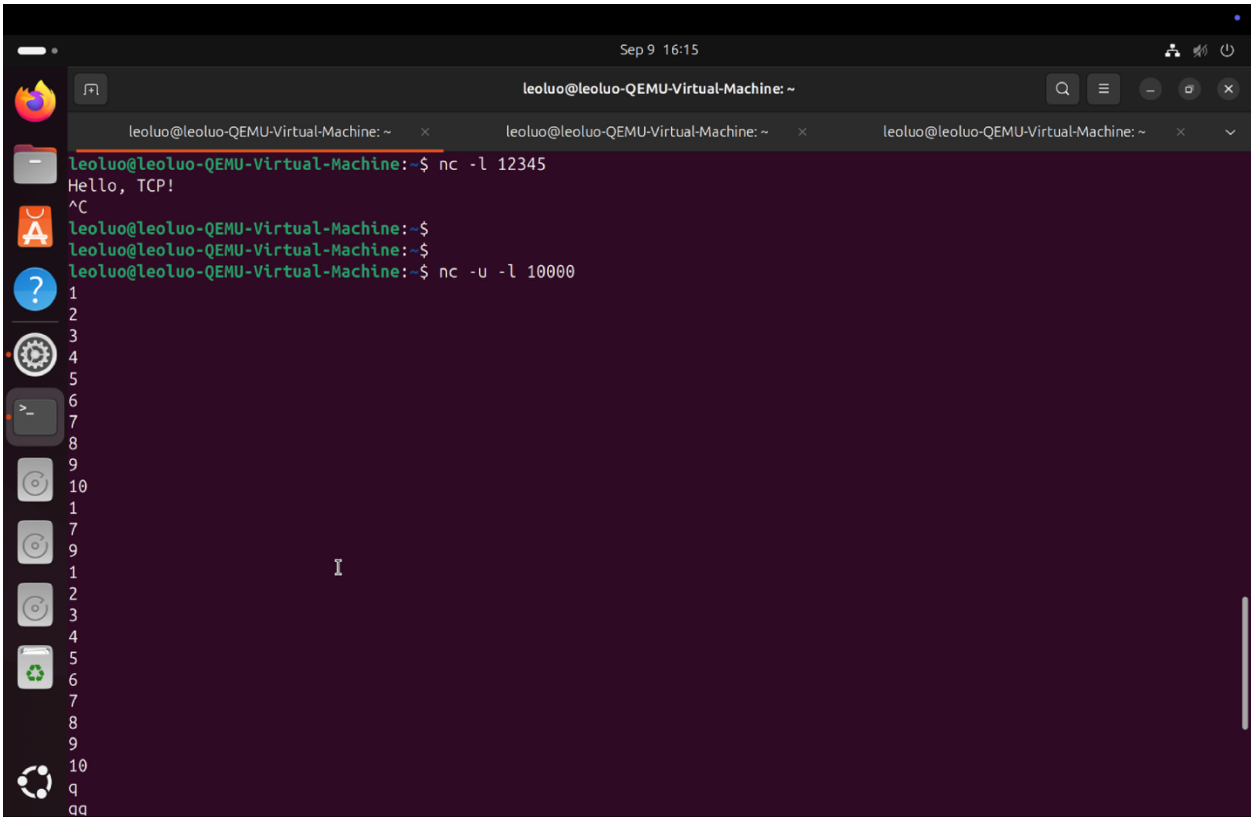
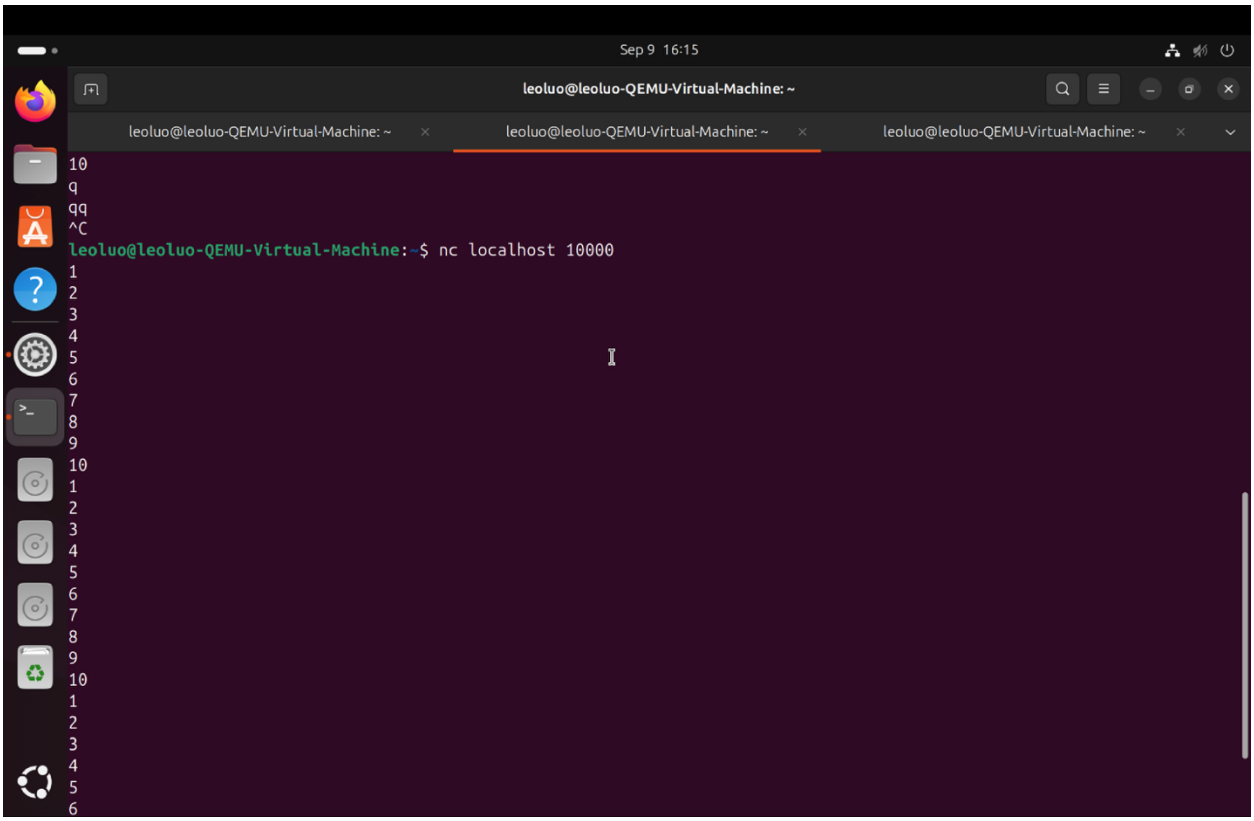
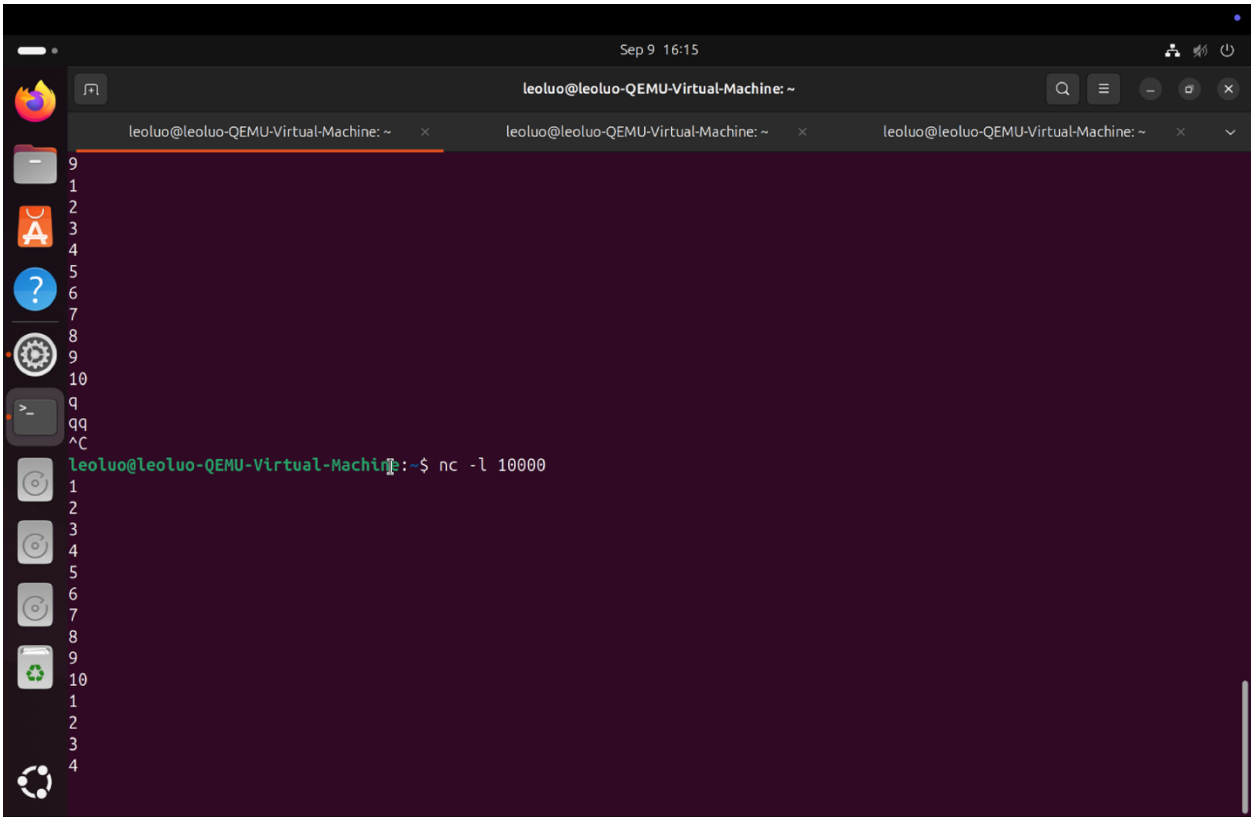
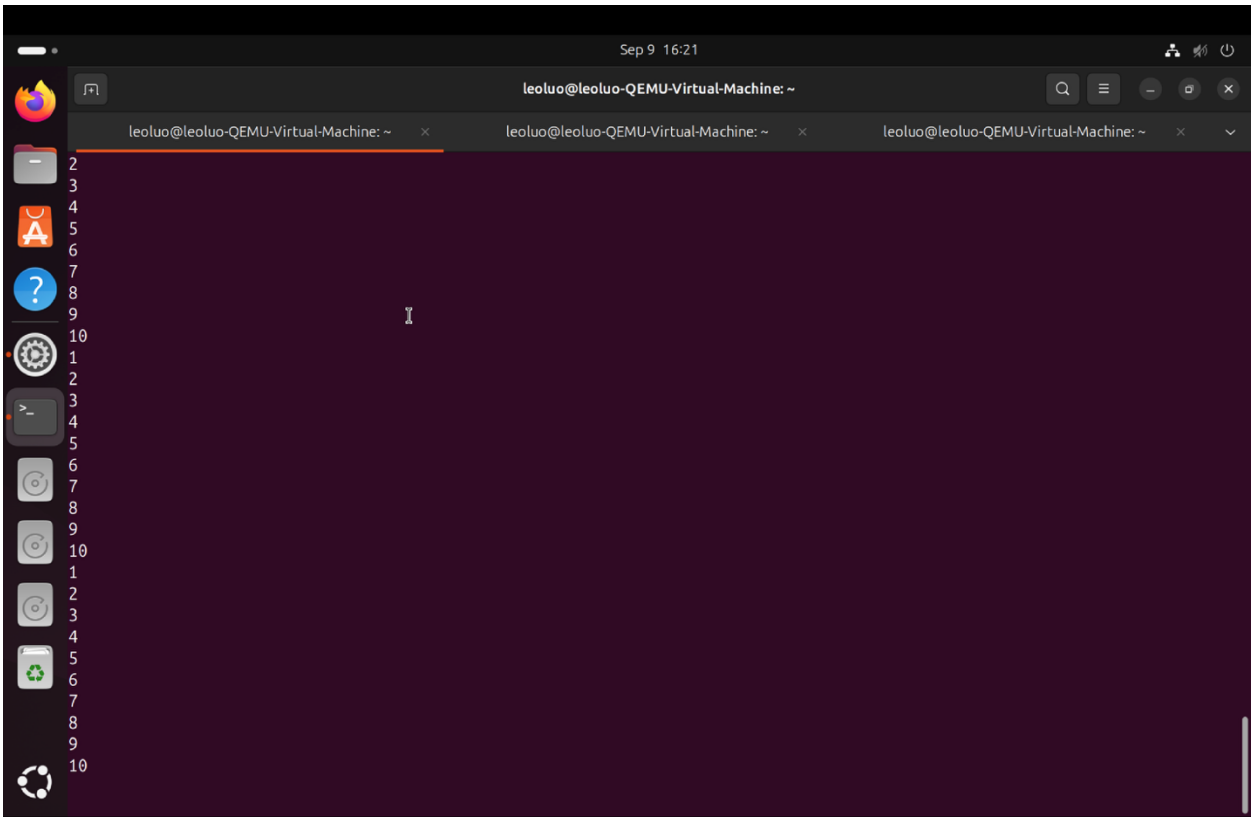
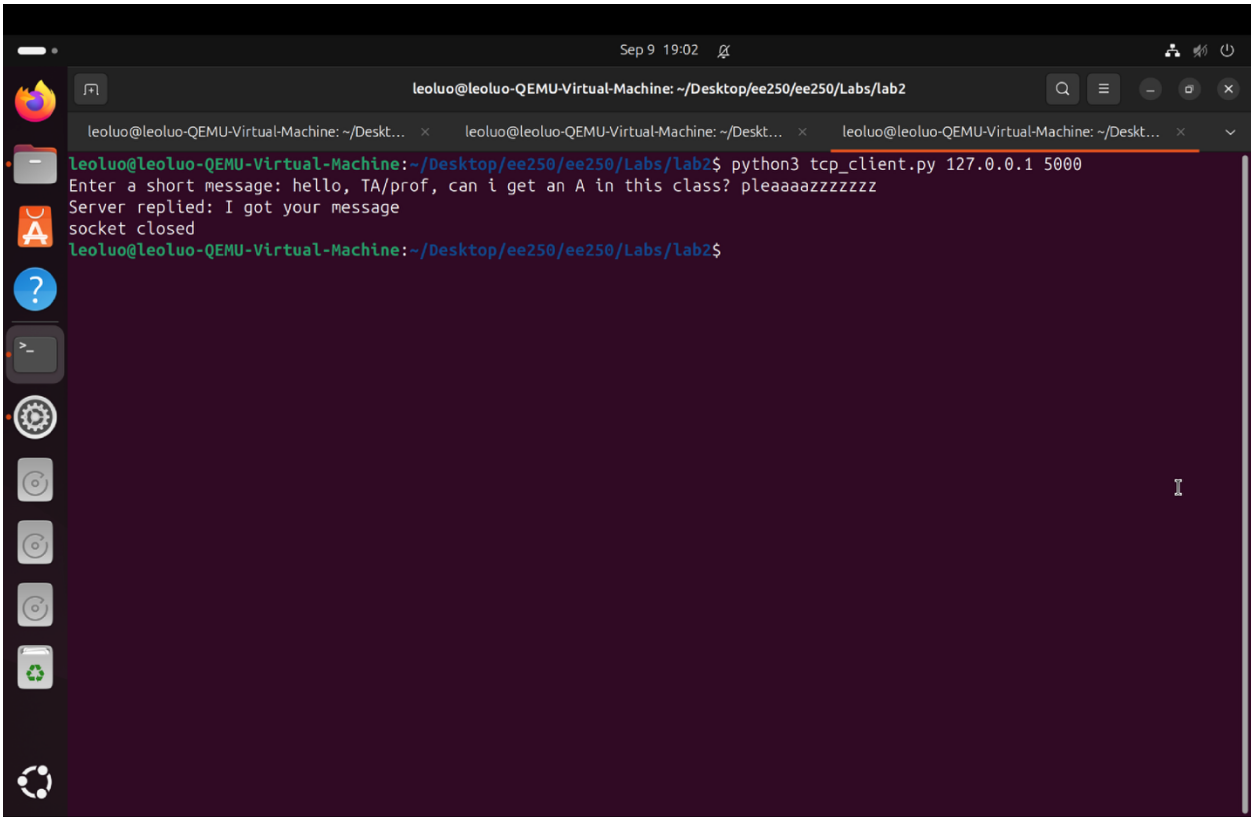

Sep 9 16:14
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
^C
leoluo@leoluo-QEMU-Virtual-Machine:~$ nc -u localhost 10000
1
2
3
4
5
6
7
8
9
10
1
2
3
4
5
6
7
8
9
10
1
2
3
4
5
6
7
8
9

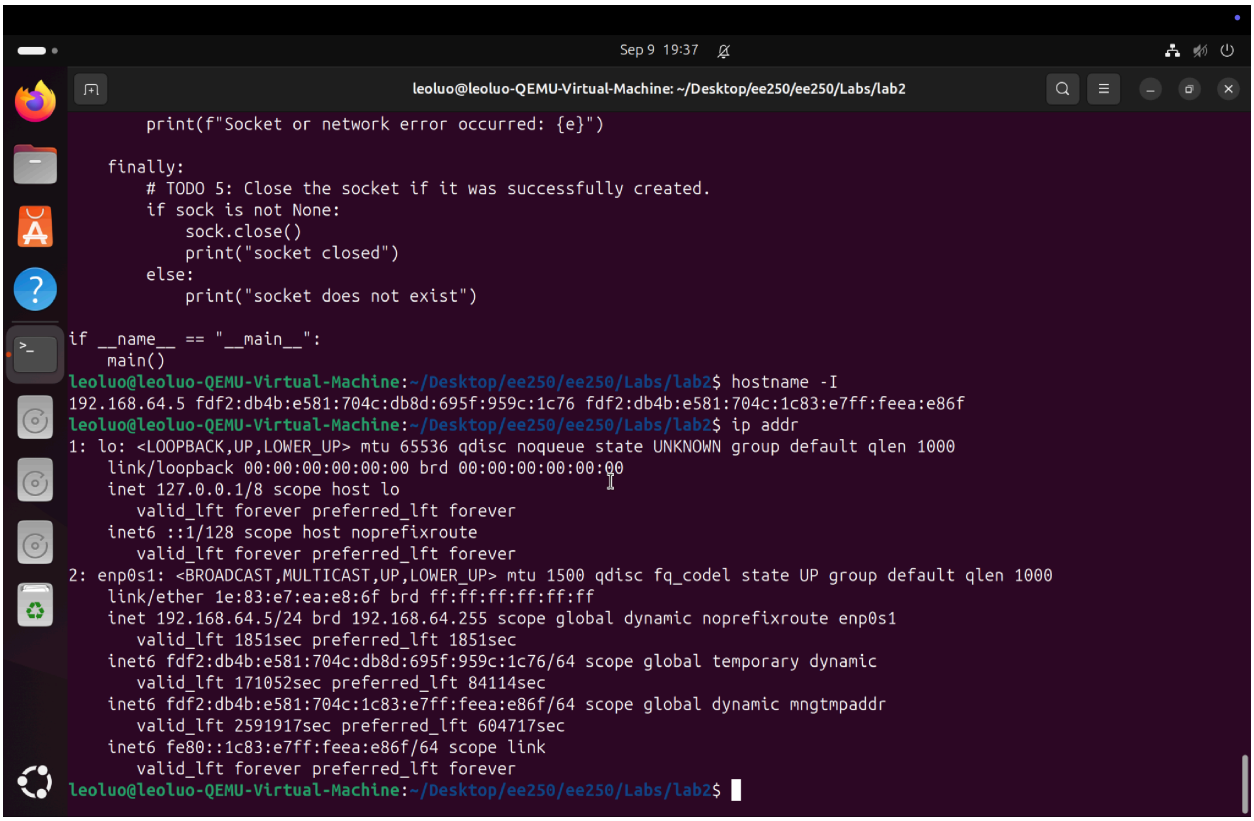

Sep 9 16:15
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine:~$ nc -l 12345
Hello, TCP!
^C
leoluo@leoluo-QEMU-Virtual-Machine:~$
leoluo@leoluo-QEMU-Virtual-Machine:~$
leoluo@leoluo-QEMU-Virtual-Machine:~$ nc -u -l 10000
1
2
3
4
5
6
7
8
9
10
1
7
9
1
2
3
4
5
6
7
8
9
10
q
qq

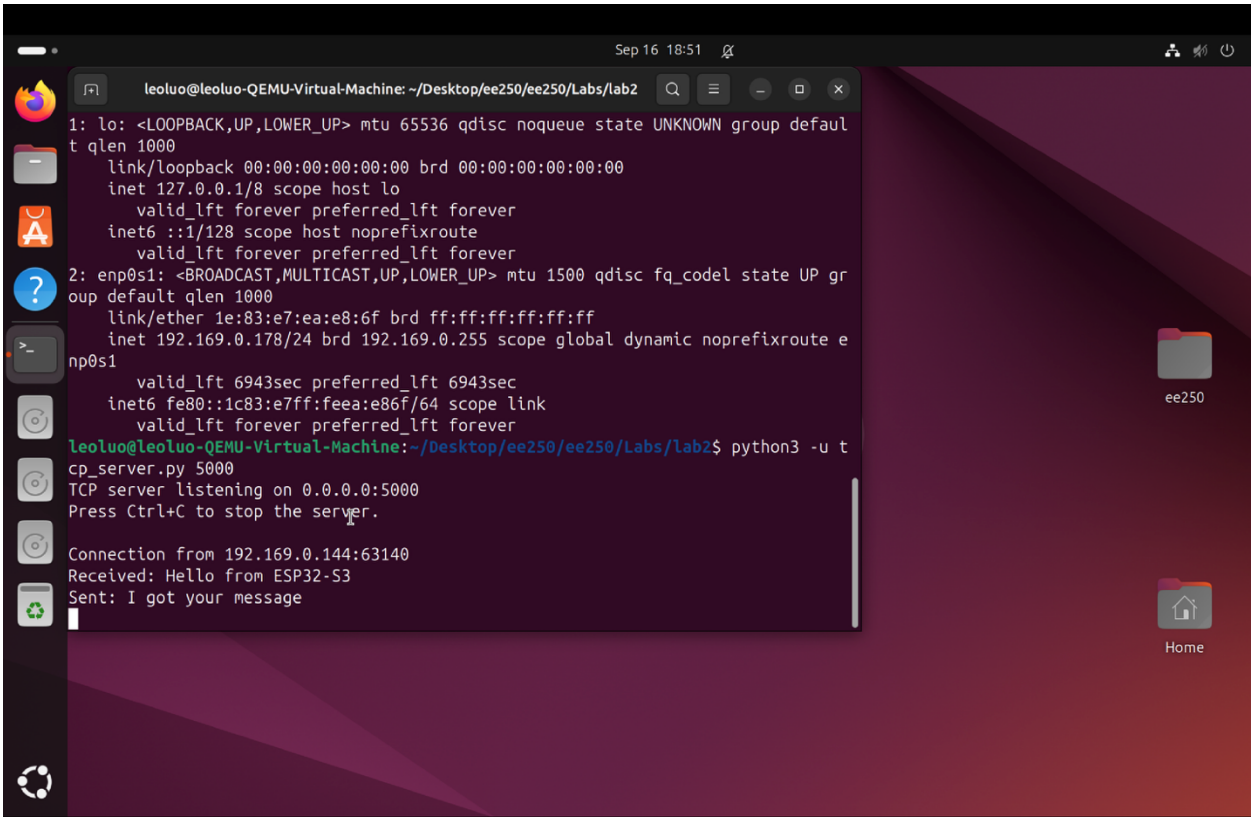

Sep 9 16:15
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
10
q
qq
^C
leoluo@leoluo-QEMU-Virtual-Machine:~$ nc localhost 10000
1
2
3
4
5
6
7
8
9
10
1
2
3
4
5
6
7
8
9
10
1
2
3
4
5
6

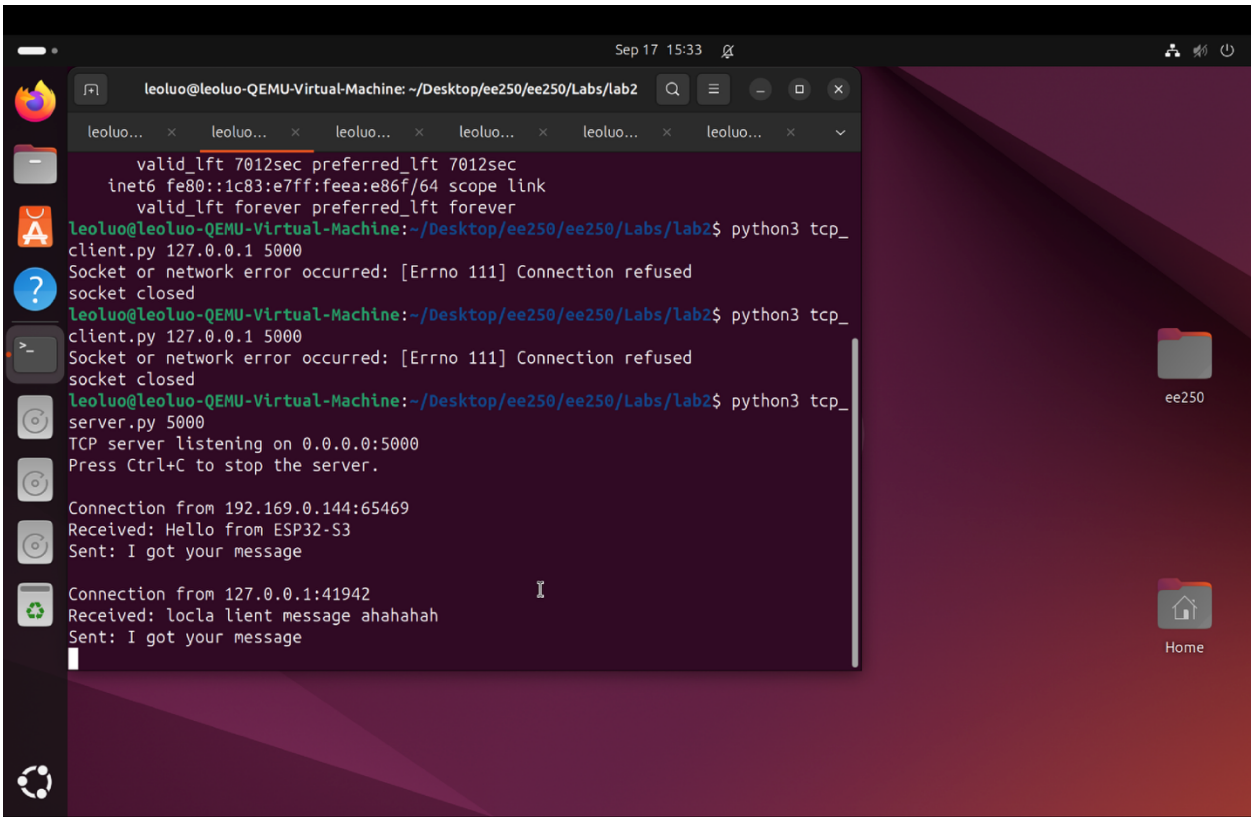

Sep 9 16:15
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
9
1
2
3
4
5
6
7
8
9
10
q
qq
^C
leoluo@leoluo-QEMU-Virtual-Machine:~$ nc -l 10000
1
2
3
4
5
6
7
8
9
10
1
2
3
4


Sep 9 16:21
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
leoluo@leoluo-QEMU-Virtual-Machine: ~
2 3 4 5 6 7 8 9 10 1 2 3 4 5 6 7 8 9 10 1 2 3 4 5 6 7 8 9 10


Sep 9 19:02
leoluo@leoluo-QEMU-Virtual-Machine: ~/Desktop/ee250/ee250/Labs/lab2
leoluo@leoluo-QEMU-Virtual-Machine: ~/Deskt...
leoluo@leoluo-QEMU-Virtual-Machine: ~/Deskt...
leoluo@leoluo-QEMU-Virtual-Machine: ~/Deskt...
leoluo@leoluo-QEMU-Virtual-Machine:~/Desktop/ee250/ee250/Labs/lab2$ python3 tcp_client.py 127.0.0.1 5000
Enter a short message: hello, TA/prof, can i get an A in this class? pleaaaazzzzzzz
Server replied: I got your message
socket closed
leoluo@leoluo-QEMU-Virtual-Machine:~/Desktop/ee250/ee250/Labs/lab2$


Sep 9 19:37
leoluo@leoluo-QEMU-Virtual-Machine: ~/Desktop/ee250/ee250/Labs/lab2
print(f"Socket or network error occurred: {e}")
finally:
TODO 5: Close the socket if it was successfully created.
if sock is not None:
sock.close()
print("socket closed")
else:
print("socket does not exist")
if __name__ == "__main__":
main()
leoluo@leoluo-QEMU-Virtual-Machine:~/Desktop/ee250/ee250/Labs/lab2$ hostname -I
192.168.64.5 fdf2:db4b:e581:704c:db8d:695f:959c:1c76 fdf2:db4b:e581:704c:1c83:e7ff:feea:e86f
leoluo@leoluo-QEMU-Virtual-Machine:~/Desktop/ee250/ee250/Labs/lab2$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 1e:83:e7:ea:e8:6f brd ff:ff:ff:ff:ff:ff
inet 192.168.64.5/24 brd 192.168.64.255 scope global dynamic noprefixroute enp0s1
valid_lft 1851sec preferred_lft 1851sec
inet6 fdf2:db4b:e581:704c:db8d:695f:959c:1c76/64 scope global temporary dynamic
valid_lft 171052sec preferred_lft 84114sec
inet6 fdf2:db4b:e581:704c:1c83:e7ff:feea:e86f/64 scope global dynamic mngtmpaddr
valid_lft 2591917sec preferred_lft 604717sec
inet6 fe80::1c83:e7ff:feea:e86f/64 scope link
valid_lft forever preferred_lft forever
leoluo@leoluo-QEMU-Virtual-Machine:~/Desktop/ee250/ee250/Labs/lab2$


Sep 16 18:51
leoluo@leoluo-QEMU-Virtual-Machine: ~/Desktop/ee250/ee250/Labs/lab2
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
link/ether 1e:83:e7:ea:e8:6f brd ff:ff:ff:ff:ff:ff
inet 192.169.0.178/24 brd 192.169.0.255 scope global dynamic noprefixroute e
np0s1
valid_lft 6943sec preferred_lft 6943sec
inet6 fe80::1c83:e7ff:feea:e86f/64 scope link
valid_lft forever preferred_lft forever
leoluo@leoluo-QEMU-Virtual-Machine:~/Desktop/ee250/ee250/Labs/lab2$ python3 -u t
cp_server.py 5000
TCP server listening on 0.0.0.0:5000
Press Ctrl+C to stop the server.
Connection from 192.169.0.144:63140
Received: Hello from ESP32-S3
Sent: I got your message
ee250
Home



Part 1

1. How did introducing 50% packet loss affect the UDP messages you observed? Explain why.

about 50% of the messages will randomly diappear.

2. How did introducing 50% packet loss affect the TCP exchange? Did the application data disappear, arrive

later, or behave differently? Explain why.

it will slow down the transmission of the message by around 50%. However, all messages are guranteed to arrive.

3. Why is localhost useful for testing applications even though no packets need to traverse the physical Wi-Fi

network?

it is a greate way to locally test UDP and TCP system without physical wifi network.

Part 2

1. Which side calls listen()/accept(), and which side calls connect()? Why?

client calls connect(), because connecting requires machine address that the client provides. And client chooses what server and what port to connect with.

Server calls listen() and accept(), because server is the one that accepts connect requests sent by clients.

and listen() is the command that enables the server to accept connect request from the clients.

2. What two pieces of information does the client need in order to connect to the server?

Server Host and Server Port

3. What is the difference between 127.0.0.1 and the VM IPv4 address you found with hostname -I or ip

addr?

one is only for local device (for esp itself) , and the other is for external device/inter device communication

4. The same TCP connection can carry data in both directions. Identify where this happens in the

client/server exchange.

socket.sendall()

socket.recv()

socket.listen()

socket.sendall()

part 3

1. Did tcp_server.py need to know that the second client was an ESP32-S3 written in C rather than a Python

program? Why or why not?

no the program does not need to know, cause eventually the data trasmitted are in binaries

2. Why can the ESP32-S3 not use 127.0.0.1 to reach the server running in the Ubuntu VM?

Because 127.0.0.1 is local device, it is pointing to the esp itself.

3. Why can a VM in NAT mode have Internet access yet still be difficult for the ESP32-S3 to contact directly?

because NAT assignes an additional IP address for the virtual machine and isolates the virtual machine from the local native mac.

However, if using bridge, the mac IP addeess willl be fowarded to VM machine.

4. What role does the TCP port number play when both the ESP32-S3 and VM already have IP addresses?

because IP address only identify which local network are you using. it does not specify which device or which program to send or receive messages.

you need a port number to identify the specific device and program.

5. What does TCP guarantee for the byte stream, and what does the application still have to define for itself?

the application still have to define its message boundaries and higher level protocols.

TCP byte stream gurantees that all messages will definitly getting sent and received. so even though amount of payloads are limited in

a certain amount of time, TCP will make sure to queue the remaining message for the next round.